## 1 - Pavages

### 1.1 Pavages en une dimension :

Le premier défit qu'on s'est donnés a été de trouver un algorithme permettant de paver une bande avec des polyominos en une dimensions. Nous avons une bande de taille n que nous devons paver avec des polyominos que fixe l'utilisateur.

Le moyen le plus simple pour modéliser des polyominos à une dimension est de les modéliser avec un nombre, qui représente la longueur du polyomino.

L'algorithme est alors très simple. Nous utilisons l'algorithme glouton avec du backtracking. Nous commençons par ranger les polyominos dans l'ordre décroissant. Ensuite, nous testons pur chaque polyomino si celui-ci peut-être ajouté tout à gauche de la bande. Si c'est le cas, on l'ajoute et on essaie de paver récursivement le reste de la bande. Si des polyominos donnés par l'utilisateur ne peut entrer, alors la bande actuelle ne peut pas être pavée. Si c'est un appel récursif, nous revenons en arrière et nous testons les autres possibilités. Sinon, la bande n'est pas pavable.

Exemple :
On doit paver _ _ _ _ _ _ avec ()()() et ()()()(), c'est à dire n = 6, p1 = 3, p2 = 4 :
On les mets dans l'ordre décroissant de taille : ()()()(), ()()()

On essaie de paver avec p2 :
()()()_ _
On doit maintenant paver _ _ :
Ni p1, ni p2 ne convient. Revenons donc en arrière et pavons avec p1 :
()()()_ _ _
On peut maintenant repaver avec p1 :
()()() ()()() est la solution !

### 1.2 Pavages en deux dimensions :

Maintenant, trouvons un algorithme permettant de paver quelque chose en deux dimensions. Nous avons une zone à paver et un nombre fixé de polyominos pour le paver.

Maintenant, nous utilisons des matrices de 1 et de 0 pour représenter les polyominos et la zone à paver.

Par exemple :
  []
[] [] []
est représenté
0 1 0
1 1 1

Pour paver, nous commençons par trouver un target. Le target est un point vide tel que toutes les lignes en bas sont pavées, et toutes les cases à sa gauche le sont également. Dans la prochaine itération, il faudra paver ce point. **Il n'y en a qu'un.**
Nous remarquons alors que pour chaque polyomino, d'après les hypothèses du target, il n'existe au plus une possibilité qui pourrait convenir. Car pas de rotation ni symétrie.
Alors nous testons pour chaque polyomino s'il est possible de placer à l'endroit indiqué. Ensuite, on continue de la même manière que dans l'algorithme en une dimension, c'est à dire que si on trouve rien, on revient en arrière pour tester tous les cas.

1.3 Pavage en dimension > 2 :

Reflexion

En réalité, l'algorithme de pavage en dimensions supérieures est très similaire que celui pour le pavage en 2 dimensions.

A la place de représenter les polyominos avec des matrices bidimensionnelles, nous les représentons avec des matrices de dimension > 2.

On nomme les indices x_1, x_2, x_3, …, x_n, avec n le nombre de dimensions. Alors les hypothèses sur le target sont que :

- Toute case avec son x_n plus petit que celui du target est pavée
- Si x_n de la case est égale de celle de target, alors elle doit être remplie si son x_n-1 est plus petit que x_n-1 du target
- …

Pour le reste, les deux algorithmes sont identiques.

1.4 Est-il possible de mettre en équation un problème de pavage ?

Nous nous sommes alors demandés s'il était possible de mettre en équation un problème de pavage.

Considérons les polyominos comme des graphes, où chaque case est reliée aux cases adjacentes.

Il nous vient alors l'envie mathématique d'étudier les caractéristiques de ce graphe. Étudions le degré de ce graphe, c'est à dire la somme des liaisons de chacun des nœuds du graphe.

Par exemple,

()()()
  ()

sera représenté

a-b-c
  |
  d

avec a, b, c et d des nœuds.

Nous appelons le degré réel (noté d_r) du polyomino le degré du graphe représentant le polyomino.

Ce polyomino par exemple a un degré réel de 1+3+1+1 = 6.

Nous appelons le degré potentiel (noté d_p) du polyomino le degré du graphe représentant le polyomino, mais en ajoutant les liaisons sortantes, comme si le polyomino avait été encastré dans un autre polyomino.

Ce polyomino par exemple a un degré potentiel de 4+4+4+4 = 12

Nous voulions alors avec cela mettre un problème de pavage en équation. On pensait avoir quelque chose de la forme :

Soit $d_p_1$, $d_r_1$ les degrés potentiels et réels du premier polyomino, $d_p_2$, $d_r_2$ ceux du second et d_p, d_r ceux de la zone à paver.

Exprimer le degré potentiel de p en fonction du d_p de p1 et du d_p de p2

Exprimer le degré réel de p en fonction du d_p de p1 et du d_p de p2

Pour enfin résoudre cette équation et obtenir les résultats.

Quand on agence deux polyominos, le degré potentiel de l'assemblage est la somme des degrés potentiels des deux polyominos. Mais le degré réel de l'assemblage est quant à lui variable en fonction de comment on agence les polyominos. Il nous était donc impossible de mettre en équation le pavage de cette manière …

## 2 – Caractérisation d'un ensemble de polyominos

Nous nous sommes donnés comme défi de caractériser avec quelques variables un ensemble de polyominos. Ici, nous essayons de caractériser plus spécifiquement la classe complète d'un polyomino.

### 2.1 Découvertes sur les degrés

Les degrés réels et potentiels ne nous ont pas servi à la caractérisation des polyominos individuels, mais nous avons fait plusieurs découvertes surprenantes dessus.

Tout d'abord, nous avons remarqué que le degré potentiel d'un polyomino vaut 4 fois son aire. En effet, chaque case du polyomino est reliée à toutes les cases adjacentes, remplies ou pas. Donc chaque case est reliée à 4 cases. La somme est juste 4 fois le nombre de cases, c'est à dire 4 fois l'aire.

Ensuite, on a remarqué que d_p – d_r vaut le périmètre du polyomino ! Effectivement, si on prend toutes les liaisons, et qu'on enlève les liaisons qui vont vers l'intérieur du polyomino, il ne reste que les liaisons qui sortent du polyomino.

Avec cela, on peut alors montrer que deux polyominos ont la même aire et le même périmètre si et seulement s'ils ont le même degré réel et potentiel.

Comment obtenir ces degrés ?

Les polyominos sont stockés comme des matrices numpy de 0 et de 1 donc il suffit de faire polyomino.sum() pour obtenir l'aire. Aire*4 pour obtenir le degré potentiel.

Pour obtenir le degré réel, on peut calculer le périmètre **ou** trouver directement avec la définition de degré réel.

Pour calculer le périmètre, il suffit de faire la convolution entre la matrice

```
0  1  0
1 -4  1
0  1  0
```

et le polyomino, et on compte ne nombre de valeures strictement positives.
(appliquer la matrice pour chaque case).
Enfin, d_p – d_r = perimetre <=> d_r = d_p – perimetre

Pour trouver le degré réel directement avec sa définition, on applique la convolution entre le polyomino et la matrice. Le 4 permettant de repérer ce qui est situé à l'intérieur du polyomino.

```
0  1  0
1  4  1
0  1  0
```

On soustrait 4 et on enlève les valeurs négatives. On termine par calculer la somme des valeurs de la matrice. Ainsi, si une case à 3 liaisons elle aura le chiffre 3.

En d'autres termes, on met une valeur positive aux cases dans le polyomino, et les autres sont égales à 0.

2.2 Utilisation des degrés dans la caractérisation d'ensembles de polyominos.

Malheureusement, nous avons vite réalisé que ces deux degrés (caractérisation) ne suffisent pas pour caractériser une classe complète. Le problème vient de la représentation en tant que graphes. Par exemple, ces deux polyominos sont identiques lorsque représentés comme des graphes :

()()
()
et
()()()

La position relative des cases – les unes par rapport aux autres - n'est pas prise en compte.

2.3 Comptage du nombre de cases étant adjacentes à 1/2/3/4 cases du polyomino

Nous avons essayé d'ajouter des valeurs qui viendraient discriminer ces polyominos identiques.

Dans le calcul du périmètre, une fois la convolution appliquée. Nous ne regardions pas les valeurs données mais seulement leur positivité. Mais étudions ces valeurs ! Cela nous donnera une information en plus

Ces quelques attributs permettaient de discriminer parfaitement les polyominos jusqu'à une aire de 6. Avec une aire de 6, cette méthode n'était plus suffisante 😔. Ces polyominos avaient les mêmes caractéristiques :

()()
  ()()
  ()()
et
  ()
  ()()
()()()

2.4 Ajout de liaisons diagonales

On change la représentation des polyominos pour lier les cases diagonalement aussi.
Le polyomino
()()()
  ()
sera alors représenté
o-o-o
 \ | /
  o

On remarque que ses degrés réels et potentiels changent !

Le changement du degré potentiel est sans importance, ce n'est plus 4 * aire mais plutôt 8 * aire. Le changement du degré réel quant à lui à un réel 😂 intérêt. Il permet par exemple de discriminer les polyominos

()()()
et
()()
()

2.5 Généralisation pour ajout de liaisons entre cases plus éloignées.

Nous pouvons aussi ajouter des liaisons entre deux cases à 2 de distance l'une de l'autre.



devient donc

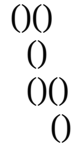


Avec le premier nœud et le dernier qui sont connectés.

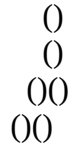
Le degré réel est encore différent !

Nous calculons tous les degrés réels en ajoutant des liaisons toujours plus longues. De cette manière, on obtient une liste de valeurs. A partir d'une certaines tailles, les degrés réels sont toujours les mêmes. On s'arrête donc à ce moment là.

Les résultats sont bon jusqu'à la taille 6, où on a par exemple ces deux polyominos considérés de la même classe complète :



et



Nous avons diminué le nombre d'erreurs !

## 3 – Étude des polyominos sécables

### 3.1 Définitions

Un polyomino est dit sécable ou binaire s'il peut être découpé en deux parties égales.

Ainsi, un polyomino ternaire/quaternaire/… représente un polyomino qui peut être découpé en 3/4/… parties égales. Ces parties doivent être **strictement égales**, aucune rotation n'est permise.

On dit qu'un polyomino est sécable horizontalement ou verticalement s'il est possible de le découper selon une droite respectivement horizontale ou verticale.

On dit qu'un polyomino est sécable n fois si ses parties une fois découpé peuvent encore une fois être découpées n-1 fois.

### 3.2 Premières propriétés

FAUX, 111|111

Tout d'abord, on remarque qu'un polyomino sécable n fois doit forcément avoir 2^n cases.

Un polyomino sécable est sécable horizontalement ou verticalement, et ne peut pas être sécable selon quelque chose d'autre qu'une droite.

Si on prend le plus petit rectangle qui peut contenir le polyomino. Soient n et m respectivement la longueur et la hauteur du rectangle. Alors le polyomino ne peut être sécable horizontalement que si n est pair et verticalement que si m est pair (condition nécessaire mais pas suffisante).

### 3.3 Algorithme pour savoir si un polyomino est sécable

(en gros, on coupe en deux et si ça marche c'est bon !)

Tout d'abord on cherche la taille du plus petit rectangle contenant le polyomino. Si n et m sont impairs, alors le polyomino est directement insécable.

Si l'un des deux est pairs, nous allons tester s'il est sécable horizontalement / verticalement en fonction de si n est pair ou m est pair. Pour ce faire, nous coupons selon la droite verticale / horizontale qui passe par le centre du polyomino et nous regardons si les deux parties obtenues sont égales.

### 3.4 Que se passe-t-il si on s'écarte de la définition ?

4 – Relier deux points selon les règles de tetris

On va voir ça plus tard